CLASE GUIADA / 90 MINUTOS

Consumir una API sin perder el hilo del proyecto

Fetch · Componentes · children · Custom hooks

La clase parte de la tienda existente y desarrolla una única funcionalidad: cargar su catálogo desde una API. Sobre ese mismo catálogo incorporamos estados de carga y error, composición visual y un custom hook.

El recorrido de esta clase

```
API de productos
  -> función que obtiene y adapta los datos
  -> estado del catálogo
  -> sección visual con children
  -> lógica extraída a useProducts
```

El detalle que se presenta al final utiliza esa misma API. La búsqueda con array local y setTimeout no forma parte de este recorrido.

Proyecto: C:/coder-react-93510. Reutilizamos ItemListContainer, ItemList, Item, Navbar y los estilos que ya tienen. Los cambios se muestran como una demostración del docente, sin consignas de participación durante la exposición.

El código es texto seleccionable, sin números de línea ni cortes de instrucciones. Las indicaciones de archivo y reemplazo están fuera de los bloques. El Markdown adjunto contiene exactamente los mismos fragmentos.

APERTURA / 0-5 MIN

Texto para presentar lo que vamos a aprender

Nuestra tienda ya tiene una estructura que funciona: un componente obtiene los productos, otro recorre la lista y otro muestra cada tarjeta. En la clase anterior usamos una promesa para representar una operación que tarda en completarse. Hoy vamos a usar esa misma idea para obtener información desde un servidor.

Vamos a conectar el catálogo a una API real. Primero veremos cómo llega la respuesta y cómo convertirla al formato que nuestras tarjetas ya saben mostrar. Después haremos que la pantalla comunique si está cargando, si ocurrió un error o si la consulta terminó sin productos.

Cuando esa parte funcione, vamos a ordenar el código que acabamos de escribir. Usaremos children para reutilizar la sección visual del catálogo y crearemos un custom hook para separar la lógica de carga de la presentación.

Al terminar, la tienda seguirá siendo reconocible, pero los datos vendrán de una API y cada archivo tendrá una responsabilidad más clara. Esa misma forma de trabajo nos servirá para construir el detalle de un producto, consultando su ID en el mismo servidor.

El objetivo de hoy es entender el recorrido completo de los datos y poder seguirlo sin mezclar la consulta, el estado y el JSX en una única responsabilidad.

Un solo ejemplo que crece durante la explicación

0-5: apertura y objetivo de la clase.

5-12: repaso del flujo de componentes existente.

12-27: respuesta de la API, fetch y adaptación de campos.

27-45: catálogo con carga, error y lista vacía.

45-57: extraer la sección visual con children.

57-77: mover la lógica conocida a useProducts.

77-85: explicar la consulta individual en la misma API.

85-90: cierre y presentación de la preentrega adaptada.

Antes de la clase, ejecutar npm run dev y comprobar el endpoint de laptops. Tener a mano los bloques completos del contenedor y del hook para incorporarlos y explicar sus partes sin transcribir cada línea en vivo.

La demostración conserva el mismo catálogo durante todo el desarrollo. No hay cambios a otra fuente de productos, ejemplos de otro dominio ni vistas temporales de contadores.

ItemCount se incluye como material de apoyo al final porque no existe todavía con ese nombre en esta copia. Su código utiliza solamente props, estado y eventos, y se utilizará en el detalle de la preentrega.

FASE 1 / 5-12 MIN

Repasar componentes desde la tienda que ya existe

Responsabilidades actuales

```
ItemListContainer -> obtiene y guarda los productos
ItemList          -> recorre el array con map
Item              -> muestra una tarjeta
Navbar            -> presenta la navegación
```

Abrimos los archivos siguiendo ese orden. El contenedor concentra la obtención de datos. ItemList recibe una lista por props y genera un Item por producto. Item recibe prod y muestra sus propiedades.

Al cambiar el origen de los datos no necesitamos reconstruir las tarjetas. Conservaremos el formato que ya usan: id, name, image, price, category, description y stock.

Item.jsx / agregar la categoría debajo del nombre

```
<p className="item-category">{prod.category}</p>
```

Esta línea completa el resumen del producto con una prop que ya existe en los datos. La descripción completa y el contador pertenecen a la futura vista de detalle.

El archivo de la clase anterior puede permanecer en el proyecto, pero al conectar la API quitaremos su import del contenedor. Desde ese momento todo el recorrido de esta clase utiliza el servidor externo.

FASE 2 / 12-27 MIN

Primero observar la respuesta de la API

Endpoint del catálogo / abrir en el navegador

```
https://dummyjson.com/products/category/laptops?limit=6
```

La consulta pide hasta seis productos de la categoría laptops. El límite no obliga a que existan seis. La cantidad real depende de los datos disponibles en la API.

Forma abreviada / ejemplo para leer la estructura

```
{
  "products": [
    {
      "id": 78,
      "title": "Laptop de ejemplo",
      "thumbnail": "https://servidor/imagen.webp",
      "price": 999.99,
      "category": "laptops",
      "description": "Información del producto",
      "stock": 7
    }
  ],
  "total": 5
}
```

La respuesta es un objeto. La lista se encuentra en products. Además, el servidor llama title al nombre y thumbnail a la imagen; nuestras tarjetas esperan name e image.

Identificar estas diferencias antes de modificar React evita dos problemas frecuentes: intentar hacer map sobre el objeto completo y mostrar propiedades que no existen.

Dos await, dos tareas diferentes

Consola del navegador / demostración del docente

```
const response = await fetch(
  "https://dummyjson.com/products/category/laptops?limit=6"
);
console.log(response.status, response.ok);
const result = await response.json();
console.log(result.products);
```

El primer await obtiene la respuesta HTTP. Con ella conocemos el status y podemos leer el body. El segundo await procesa el contenido JSON para obtener los datos que usaremos en JavaScript.

Un 404 o un 500 no hace que fetch rechace automáticamente su promesa. Si hubo una respuesta HTTP, debemos comprobar response.ok. Esta propiedad es true para los estados entre 200 y 299.

La comprobación antes de leer los datos

```
if (!response.ok) {
  throw new Error("No pudimos cargar los productos.");
}
```

El error lanzado será atendido por catch en el contenedor. Allí también pueden llegar fallos de conexión o de lectura de JSON. El usuario verá un mensaje en la interfaz en lugar de depender de la consola.

Una función pequeña para hablar con el servidor

Creamos services/productsApi.js para concentrar la petición y el cambio de nombres. No utiliza hooks: es una función async que devuelve una promesa con un array adaptado.

Crear src/services/productsApi.js / archivo completo

```
const API_URL =
  "https://dummyjson.com/products/category/laptops?limit=6";
export async function getProductsFromApi() {
  const response = await fetch(API_URL);
  if (!response.ok) {
    throw new Error("No pudimos cargar los productos.");
  }
  const result = await response.json();
  if (!Array.isArray(result.products)) {
    throw new Error("La respuesta no contiene una lista de productos.");
  }
  return result.products.map((product) => ({
    id: product.id,
    name: product.title,
    image: product.thumbnail,
    price: product.price,
    category: product.category,
    description: product.description,
    stock: product.stock
  }));
}
```

El map transforma cada producto de la API al formato que ya consume Item. Conservamos description y stock porque se usarán al mostrar información completa.

La función no decide qué mensaje o qué componente mostrar. Su responsabilidad termina al devolver los datos o rechazar la promesa con un error.

El estado representa lo que está ocurriendo

Estados del catálogo

```
data = []       -> los productos disponibles para renderizar
loading = true  -> la consulta todavía no terminó
error = null    -> todavía no hay un error para mostrar
```

Mientras esperamos, mostramos Cargando productos. Si falla la consulta, mostramos el error. Si la respuesta es correcta y la lista está vacía, mostramos No hay productos disponibles. Solo mostramos tarjetas cuando llegaron productos.

El bloque finally finaliza la carga tanto si la consulta funciona como si falla. No usamos la longitud inicial del array para decidir que el catálogo está vacío antes de que termine la petición.

El efecto incluye una limpieza sencilla. Una variable active vale true mientras esa ejecución puede actualizar la pantalla. La limpieza la cambia a false para ignorar resultados que llegan tarde.

Esta variable no es otro estado y no cancela la petición. Solo evita utilizar los resultados de una ejecución del efecto que ya fue limpiada.

En la página siguiente reemplazamos el contenedor completo. Se conserva ItemList y se elimina el import de la función simulada anterior.

La consulta se integra al componente existente

ItemListContainer.jsx / reemplazar contenido completo

```jsx
import { useEffect, useState } from "react";
import { getProductsFromApi } from "../../services/productsApi";
import ItemList from "../ItemList/ItemList";
import "./itemlistcontainer.css";
const ItemListContainer = ({ greeting }) => {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    let active = true;
    const loadProducts = async () => {
      try {
        const products = await getProductsFromApi();
        if (active) setData(products);
      } catch (error) {
        if (active) setError(error.message);
      } finally {
        if (active) setLoading(false);
      }
    };
    loadProducts();
    return () => { active = false; };
  }, []);
  if (loading) return <p>Cargando productos...</p>;
  if (error) return <p role="alert">{error}</p>;
  if (data.length === 0) return <p>No hay productos disponibles.</p>;
  return (
    <div className="item-list-container">
      <h2 className="container-title">{greeting}</h2>
      <ItemList data={data} />
    </div>
  );
};
export default ItemListContainer;
```

La función async se declara dentro del efecto y se ejecuta con loadProducts(). El callback de useEffect no es async, porque su retorno queda reservado para la limpieza.

Comprobar el comportamiento sin cambiar de ejemplo

Primero, la carga. Desde Network en DevTools, el docente limita la velocidad de conexión y recarga. El texto Cargando productos aparece mientras se espera. Después se restablece la velocidad normal.

Después, el error HTTP. Se cambia temporalmente API_URL a https://dummyjson.com/products/99999999 y se recarga. response.ok es false y la interfaz muestra el mensaje de error. Luego se restaura la URL del catálogo.

Por último, la lista vacía. Se utiliza temporalmente https://dummyjson.com/products/search?q=zzznoproduct93510. Si el servidor responde 200 con products vacío, la interfaz muestra No hay productos disponibles. Al terminar se vuelve al endpoint de laptops.

Son tres comprobaciones del mismo flujo de carga. Un error de servidor y un resultado vacío no significan lo mismo; por eso se representan con estados diferentes.

El proyecto contiene StrictMode en main.jsx. En desarrollo React ejecuta un ciclo adicional de preparación y limpieza de efectos. Pueden verse dos peticiones; no hace falta quitar StrictMode. La limpieza que agregamos permite ignorar el resultado de la ejecución anterior.

Con [] el efecto se prepara al montar el componente. No significa que solo pueda ejecutarse una vez durante toda la vida de la aplicación.

FASE 4 / 45-57 MIN

children aparece para resolver una necesidad visual

El catálogo ya obtiene y muestra datos. Ahora ordenamos su estructura visual: una sección con título y contenido. Esa estructura no necesita conocer cómo se hizo el fetch.

Crear src/components/Section/Section.jsx / archivo completo

```
import "../ItemListContainer/itemlistcontainer.css";
const Section = ({ title, children }) => {
  return (
    <section className="item-list-container">
      <h2 className="container-title">{title}</h2>
      {children}
    </section>
  );
};
export default Section;
```

title llega como una prop con nombre. children contiene lo que colocamos entre la apertura y el cierre de Section. Reutilizamos las clases y el CSS del catálogo para conservar el diseño actual.

ItemListContainer.jsx / agregar este import

```
import Section from "../Section/Section";
```

Reemplazar solo el return final del contenedor

```
return (
  <Section title={greeting}>
    <ItemList data={data} />
  </Section>
);
```

ItemList es el contenido de Section. Section muestra ese contenido mediante {children}; si esa expresión no está en su JSX, solo veremos el título.

El marco y los datos cumplen trabajos distintos

La composición del catálogo

```
ItemListContainer
  -> Section: presenta el título y el marco
       -> ItemList: muestra la lista recibida
            -> Item: presenta cada producto
```

Section no tiene que recibir una prop por cada característica de un producto. Recibe el título y el contenido ya construido. El contenedor sigue teniendo acceso a los datos y decide qué colocar dentro.

Esto también nos permitirá usar la misma Section alrededor de los mensajes de carga y error. No crearemos otra tarjeta ni otro ejemplo de pantalla para demostrar children: lo aplicamos al catálogo que ya estamos desarrollando.

La palabra contenedor puede referirse a una responsabilidad visual o de datos. En esta clase los nombres son concretos: Section organiza la sección; ItemListContainer coordina el catálogo.

En el próximo paso extraeremos los estados y el efecto. El JSX seguirá siendo responsabilidad del contenedor, incluido lo que envía como children.

FASE 5 / 57-77 MIN

Un custom hook extrae el código que ya funciona

El contenedor reúne dos trabajos: administrar la consulta y decidir qué mostrar. Creamos useProducts para mover los estados y el efecto que acabamos de explicar, sin modificar su comportamiento.

Separación después de la extracción

```
productsApi.js -> fetch y adaptación de los productos
useProducts    -> productos, carga, error y efecto
ItemListContainer -> contenido de la sección
Section        -> título y children
ItemList / Item -> lista y tarjetas
```

Un custom hook es una función que utiliza hooks y cuyo nombre comienza con use. Se llama en el nivel superior del componente, antes de los retornos y sin condiciones.

Para esta clase basta con useProducts. No necesitamos un hook genérico que acepte cualquier respuesta, autenticación o configuración. Su trabajo está centrado en nuestro catálogo.

El hook retorna valores, no elementos de la interfaz. Compartir su código no significa compartir el estado: si se llama desde dos componentes, cada llamada tendrá sus propios estados y peticiones.

Mismos estados, mismo efecto, otro archivo

Crear src/hooks/useProducts.js / archivo completo

```
import { useEffect, useState } from "react";
import { getProductsFromApi } from "../services/productsApi";
export const useProducts = () => {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    let active = true;
    const loadProducts = async () => {
      try {
        const result = await getProductsFromApi();
        if (active) setProducts(result);
      } catch (error) {
        if (active) setError(error.message);
      } finally {
        if (active) setLoading(false);
      }
    };
    loadProducts();
    return () => { active = false; };
  }, []);
  return { products, loading, error };
};
```

Aquí reconocemos el mismo useState, try/catch/finally y la misma limpieza. La única diferencia es que ahora retornamos products, loading y error para que el componente los use.

La URL está fija en la función de consulta. Si en otra clase se incorpora una categoría o un ID variable, habrá que revisar dependencias y reiniciar los estados de cada consulta. No agregamos esa variante en este primer hook.

El contenedor queda dedicado a la interfaz

ItemListContainer.jsx / versión final completa

```
import { useProducts } from "../../hooks/useProducts";
import ItemList from "../ItemList/ItemList";
import Section from "../Section/Section";
const ItemListContainer = ({ greeting }) => {
  const { products, loading, error } = useProducts();
  return (
    <Section title={greeting}>
      {loading && <p>Cargando productos...</p>}
      {!loading && error && <p role="alert">{error}</p>}
      {!loading && !error && products.length === 0 && (
        <p>No hay productos disponibles.</p>
      )}
      {!loading && !error && products.length > 0 && (
        <ItemList data={products} />
      )}
    </Section>
  );
};
export default ItemListContainer;
```

La Section conserva el título mientras cambia su contenido. El mismo children puede contener un mensaje de carga, un error, un aviso de lista vacía o ItemList.

products comienza como un array y por eso podemos usar length. No hace falta copiar los datos del hook a otro useState: duplicaríamos la misma información.

El catálogo sigue mostrando los datos de la misma API. La extracción cambió la organización del código, no la fuente de datos ni la función de cada tarjeta.

FASE 6 / 77-85 MIN

El detalle consulta al mismo servidor

La continuación del catálogo será mostrar un producto completo. Para eso no volvemos a otra fuente de datos: DummyJSON también permite consultar un producto por su identificador.

Dos operaciones sobre la misma API

```
Listado:
GET https://dummyjson.com/products/category/laptops?limit=6
Respuesta: objeto con products, que contiene un array.
Detalle:
GET https://dummyjson.com/products/78
Respuesta: el objeto del producto directamente.
```

El docente abre un producto usando el ID de una de las laptops que acaba de mostrar el catálogo. El número 78 es un ejemplo; se utiliza un ID existente obtenido del listado.

La diferencia importante es la forma de la respuesta: en el listado usamos result.products y map; en el detalle tenemos un único objeto y no hacemos map sobre él.

En ambos casos conservamos el mismo formato interno: name sale de title e image de thumbnail. El detalle puede mostrar description y stock, además del resumen que ya muestra Item.

Una función async getProductById(productId) puede realizar esa petición y devolver una promesa. El ID se recibe como parámetro: no se escribe un producto fijo dentro de su implementación.

CIERRE / 85-90 MIN

Qué quedó construido y cómo continúa el proyecto

Hoy conectamos el catálogo a una API, representamos carga, error y lista vacía, reutilizamos una sección mediante children y extraímos la lógica de consulta a useProducts.

Cada cambio apareció después de una necesidad concreta. Primero necesitábamos obtener los datos; luego comunicar la espera y los fallos; después separar el marco visual y la lógica de carga.

Archivos de la implementación final de clase

```
src/services/productsApi.js
src/hooks/useProducts.js
src/components/Section/Section.jsx
src/components/ItemListContainer/ItemListContainer.jsx
src/components/ItemList/ItemList.jsx
src/components/Item/Item.jsx
```

ItemList se mantiene. Item solo suma la categoría. App sigue montando Navbar e ItemListContainer como antes; no necesita una vista temporal de prueba para esta clase.

Texto de cierre: nuestra tienda ya sabe pedir productos y comunicar el resultado de esa consulta. La próxima ampliación conserva ese recorrido, pero en lugar de mostrar una lista va a mostrar toda la información de un producto obtenido por su ID.

Detalle de producto conectado a la misma API

Consigna adaptada: crear getProductById(productId) dentro del servicio de productos. Debe consultar https://dummyjson.com/products/ID, revisar response.ok y devolver el producto adaptado a name, image, price, category, description y stock.

Crear ItemDetailContainer para ejecutar esa función y administrar producto, carga y error. Comenzar el producto en null. Mientras espera, mostrar carga; si falla, mostrar error; cuando llega, pasarlo por props a ItemDetail.

ItemDetail presenta imagen, nombre, precio, categoría, descripción completa y stock. Reutiliza ItemCount con el stock del producto. El ID puede recibirse por props o definirse temporalmente en el contenedor; todavía no se necesita Router.

Responsabilidades de la entrega

```
getProductById(productId) -> consulta y adapta un producto
ItemDetailContainer       -> administra la promesa y los estados
ItemDetail                -> presenta el producto completo
ItemCount                 -> limita la cantidad según el stock
```

Comprobar productos distintos, un ID inexistente y stock cero. Mantener separados los archivos y no concentrar el diseño completo en el contenedor. La función debe ser dinámica y no retornar siempre el mismo producto.

Nota docente sobre el material oficial: la consigna original utiliza un array y setTimeout. En esta versión se adapta el origen de los datos a la API para conservar el hilo de la clase. Se mantienen promesas, búsqueda por ID, separación de responsabilidades y stock. Esta adaptación debe reflejarse en la consigna que se entregue.

APOYO PARA LA PREENTREGA / FUERA DE LA EXPOSICIÓN

ItemCount con los conocimientos que ya tienen

Esta copia del proyecto tiene un contador de ejemplo, pero todavía no un ItemCount que reciba stock. Se proporciona este componente de apoyo para poder reutilizarlo en el detalle. Utiliza props, estado y eventos.

Crear src/components/ItemCount/ItemCount.jsx / apoyo

```
import { useState } from "react";
const ItemCount = ({ stock }) => {
  const [quantity, setQuantity] = useState(0);
  const increase = () => {
    if (quantity < stock) setQuantity(quantity + 1);
  };
  const decrease = () => {
    if (quantity > 0) setQuantity(quantity - 1);
  };
  return (
    <div>
      <p>Disponibles: {stock}</p>
      <button onClick={decrease} disabled={quantity === 0}>
        -
      </button>
      <span aria-live="polite"> {quantity} </span>
      <button onClick={increase} disabled={quantity >= stock}>
        +
      </button>
    </div>
  );
};
export default ItemCount;
```

Dentro de ItemDetail / cuando el producto ya está disponible

```
<ItemCount key={prod.id} stock={prod.stock} />
```

La cantidad comienza en cero, nunca baja de cero y no supera stock. La key con el ID permite reiniciar el contador si cambia el producto que se está mostrando.

CONSULTA DOCENTE

Mantener el alcance y la coherencia

Durante la demostración se desarrolla un solo catálogo. Las comprobaciones de carga, error y lista vacía las realiza el docente. No se incluyen trabajos en parejas, desafíos, votaciones ni ejercicios de participación dentro de los 90 minutos.

No se implementa una segunda obtención de datos con promesas simuladas. El archivo anterior puede permanecer en el repositorio, pero los componentes de esta clase y el detalle propuesto utilizan la API.

La preentrega se comunica como una actividad posterior. No se entrega resuelta la vista ItemDetail ni su contenedor. ItemCount se proporciona porque era un requisito previo ausente en el proyecto.

Los fragmentos completos indican cuándo reemplazan la versión anterior. Al finalizar, usar el contenedor final y useProducts; no conservar los estados duplicados de las versiones intermedias.

React: useEffect y StrictMode

React: custom hooks

React: props y children

MDN: uso de fetch

DummyJSON: listado y detalle de productos